

Design note

Project: Spoke (LLD practice platform)

Author: Shubhangini

Repo: <https://github.com/shubhangini67/spoke-llid-practice>

Live: <https://spoke-llid-practice.vercel.app>

How I modelled the platform. Screenshots and diagrams: [repo README](#).

What I shipped

Pick one of five problems, lock clarifying questions, fill types + relationships, walk a use case, submit, read a review, then revise or take the follow-up. Follow-up is a new attempt linked to the old one. I do not edit history.

Problems: Vending Machine, Parking Lot, Meeting Room Scheduler, In-memory Cache, Elevator. Each has a scenario, requirements, constraints, clarifying questions that can turn extra capabilities on, and a follow-up.

One Next.js process. One learner. No login. I tried to model the platform the same way I would model Parking Lot on a whiteboard: a few types, clear ownership, seams I can swap later.

Practice loop

Catalog → brief → start (draft) → studio with autosave → submit → evaluating → evaluated (or evaluation_failed) → review → revise / follow-up → history / compare.

Submit is always clickable. Validation is `validateForSubmit` on the design: two named types with responsibilities, every question locked, three walkthrough beats that only mention real types, at least one relationship, assumptions, rejected alternative. Fail that and you get HTTP 400, stay in draft.

I almost disabled the button. Then the rule would only exist in React. Wrong for an LLD assignment.

LLD of the platform

`src/domain` does not import Next or `fs`.

Problem has **Capability** objects. A capability is something the design should be able to do, not a required class name. "Compatible spot assignment" is a capability. `ParkingSpot` is not. Some

capabilities only count if you picked a certain clarification (multi-floor, cards, a lift bank, recurring meetings).

Attempt is the aggregate. It owns a **Design**, and after scoring an **Evaluation**. Status changes are methods on Attempt (`submit`, `beginEvaluation`, `complete`, `fail`, `retryEvaluation`). Illegal jumps throw `InvalidStateTransition`.

Design is what you submit: clarifications, types, relationships, walkthrough, assumptions, rejected path, optional notes/code.

Evaluation is a value object: dimensions, coverage with evidence, strengths, concerns, alternatives, interviewer questions, next-attempt focus, optional delta vs parent. Once written, we don't patch it. Revise instead.

PracticeService is the only use-case class. API routes map errors to 400/404/409. They don't score anything.

Ports: `ProblemRepository`, `AttemptRepository`, `Evaluator`, `LlmClient`, `Clock`, `IdGenerator`.

If I add a code-based evaluator later, it still does submit → evaluate on the same Attempt. If I parse a diagram later, it should produce a Design. The aggregate stays.

Attempt states: draft → submitted → evaluating → evaluated. From evaluating it can go to `evaluation_failed`; retry puts it back to evaluating. Revise = new Attempt with `parentAttemptId`.

Evaluation

Always runs (deterministic):

- Coverage from signals in names, responsibilities, methods, walkthrough, defense. Two hits = covered. Test: `Stall` still covers assignment.
- Structure: god class, vague responsibility, isolated types, inheritance-only, concrete cluster with no interface.
- Walkthrough: enough beats, more than one actor, a real verb, not just A talking to A.
- Scope: if you said several floors, floors have to show up.
- Defense: assumption + rejected path with a reason.

Dimensions: scope, coverage, collaboration, cohesion, extensibility, defense. UI shows a band more than the integer.

LLM, if a key is present: prose only. Extra strengths/concerns, alternatives with "fits when", questions, next focus. Prompt says do not invent a class list. Unknown names stripped. About 10s timeout.

If something breaks:

What	Status	UI
No key / timeout / garbage JSON	evaluated, degraded=true	still a full checker review
evaluate() throws	evaluation_failed	Retry on the same attempt
Invalid design	draft	domain message

No queue. The states are already there if evaluation moves off the request later.

Trade-offs

Structured studio vs a canvas vs a Java IDE: I wanted feedback I can compare across attempts. You cannot draw freely and you cannot compile. A parser can sit behind the same Evaluator later.

Capabilities vs a hidden answer key: that was the assignment. Signal matching is leaky. I show evidence instead of hiding `ParkingSpot`.

Making clarify + walkthrough required: extra friction. Without them this is a class dump with a score.

Scoring on the request: no scale. Timeout + fallback is enough for five problems.

JSON file vs sqlite: the port is `AttemptRepository`. A file is honest for one user.

No auth: design demo, not sessions.

Answers to the brief

What does the learner need to provide? Enough that a reviewer can point at a type and ask why it knows that, plus the scope they locked and one narrated use case. Missing those, feedback is guessing.

Useful feedback when two designs are both fine? Capabilities, seams, evidence, alternatives as "this shape fits when", questions, follow-up. Never a required name.

Deterministic vs LLM? Checker owns scores. Model owns the writeup when it is available.

Another evaluator or format later? `Evaluator` and `AttemptRepository`. Design is the evidence object. A future code submission can extract the same shape.

If evaluation takes time or fails? Spinner on the same request. Degrade if the model is down. Fail evaluation, not the submission, on a crash. Retry is one method on `Attempt`.